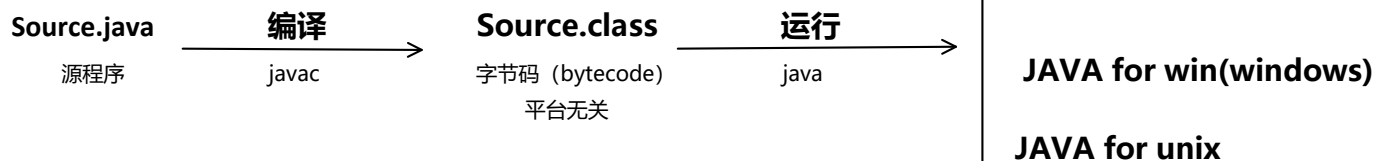


JAVA

2023年3月25日 21:33

JAVA程序的编译与运行



JRE(The Java Runtime Emironment)

JRE = JVM+API+(Lib)

JRE运行程序时的三项主要功能:

加载代码: 由class loader完成;

校验代码: 由byrecode verifier完成;

执行代码: 由runtime interpreter完成;

容量小的类型可以自动转化成容量大的类型, long类型不能自动转化成int

特例: 整形常量是int类型, 但是可以自动转成:byte/short/char 只要不超过对应类型的表数范围

方法中定义的是形参, 调用时实参, return两个作用: 返回值, 结束方法运行

语句块:

```
P S V main{
    Int n;
    Int a;
{
    Int k;
    Int n; 编译错误: 不能重复定义变量n
}变量k的作用域到此为止
```

增强For

2023年3月30日 8:47

增强For:

for(集合/数组的数据类型 变量名 : 集合名/数组名){

}

例: public static void main(String[] args) {
int values [] = new int[]{18,62,68,82,65,9};
int max = 0;
for(int eacj : values){
if (eacj > max){
max = eacj;
}
}
System.out.println(max);
}

构造

2023年3月30日 8:48

构造方法重载的条件:

- 1.形参列表不同的含义:形参类型,形参个数,形参顺序不同
- 2.只有返回值不同不够成方法的重载
如: `int a(string str){}` 与 `void a(string str){}` 不构成方法重载
- 3.只有形参的名称不同,不构成方法的重载
如: `int a(string str){}` 与 `int a(string s){}` 不构成方法重载

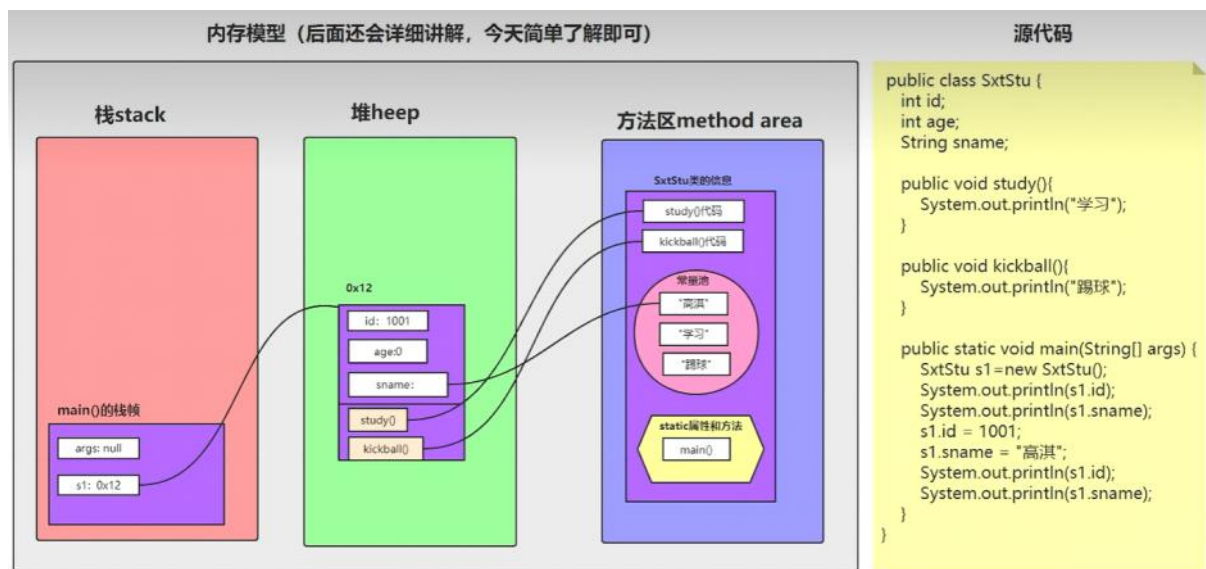
构造器四个要点:

构造器通过new关键字调用

构造器虽然有返回值,但是不能定义返回值类型(返回值的类型肯定是本类),不能在构造器里使用return返回某个值。

如果我们没有定义构造器。则编译器会自动定义一个无参的构造方法。如果已经定义则编译器不会自动添加。

构造器的方法名必须和类型一致!



栈:

- 1.栈描述的是方法执行的内存模型。每个方法被调用都

- 会创建一个栈帧(储存局部变量, 操作数、方法出口等)
- 2.JVM为每个线程创建一个栈, 用于存放该线程执行方法的信息 (实际参数、局部变量等)
- 3.栈属于线程私有, 不能实现线程间的共享!
- 4.栈的存储特性是“先进后出, 后进先出”
- 5.栈是由系统自动分配, 速度快! 栈是一个连续的内存空间

堆:

- 1. 堆用于存储创建好的对象和数组 (数组也是对象)
- 2. JVM只有一个堆, 被所有线程共享
- 3. 堆是一个连续的内存空间, 分配灵活, 速度慢!
- 4. 堆被所有的线程所共享, 在堆上的区栈会被垃圾回收器做进一步划分, 列如新生代, 老年代的划分。

方法区 (也是堆) 特点如下:

- 1. 方法区是JAVA虚拟机规范, 可以有不同的实现。
 - i. JDK7以前是“永久代”
 - ii. JDK7部分去除“永久代”, 静态变量, 字符串常量池都挪到了堆内存中
 - iii. JDK8是“元数据空间”和堆结合起来
- 2.JVM只有一个方法区, 被所有线程共享!
- 1. 方法区实际也是堆, 只是用于存储类, 常量相关的信息!
- 2. 用来存放程序中永远是不变成唯一的内容。(类信息[class对象反射机中重点讲述], 静态变量, 字符串常量等)
- 3. 常量池主要存放: 如文本字符串, final常量值。

类

2023年3月30日 8:49

类的三重成员：属性field、方法 method、构造器constructor

类的属性：又叫做静态属性。

如果一个属性的，每个“英雄”都不一样，比如name，这样的属性就应该设计为对象属性，因为它是跟着对象走的，每个对象的name都是不同的

如果一个属性，所有的英雄都共享，都是一样的，那么就应该设计为类属性。比如血量上限，所有的英雄的血量上限都是9999，不会因为英雄不同，而取不同的值。这样的属性，就适合设计为类属性。

Static int x;

对象属性：又叫做实例属性，非静态属性

如果一个属性声明成类属性，那么所有的对象，都共享这么一个值

垃圾回收相关算法

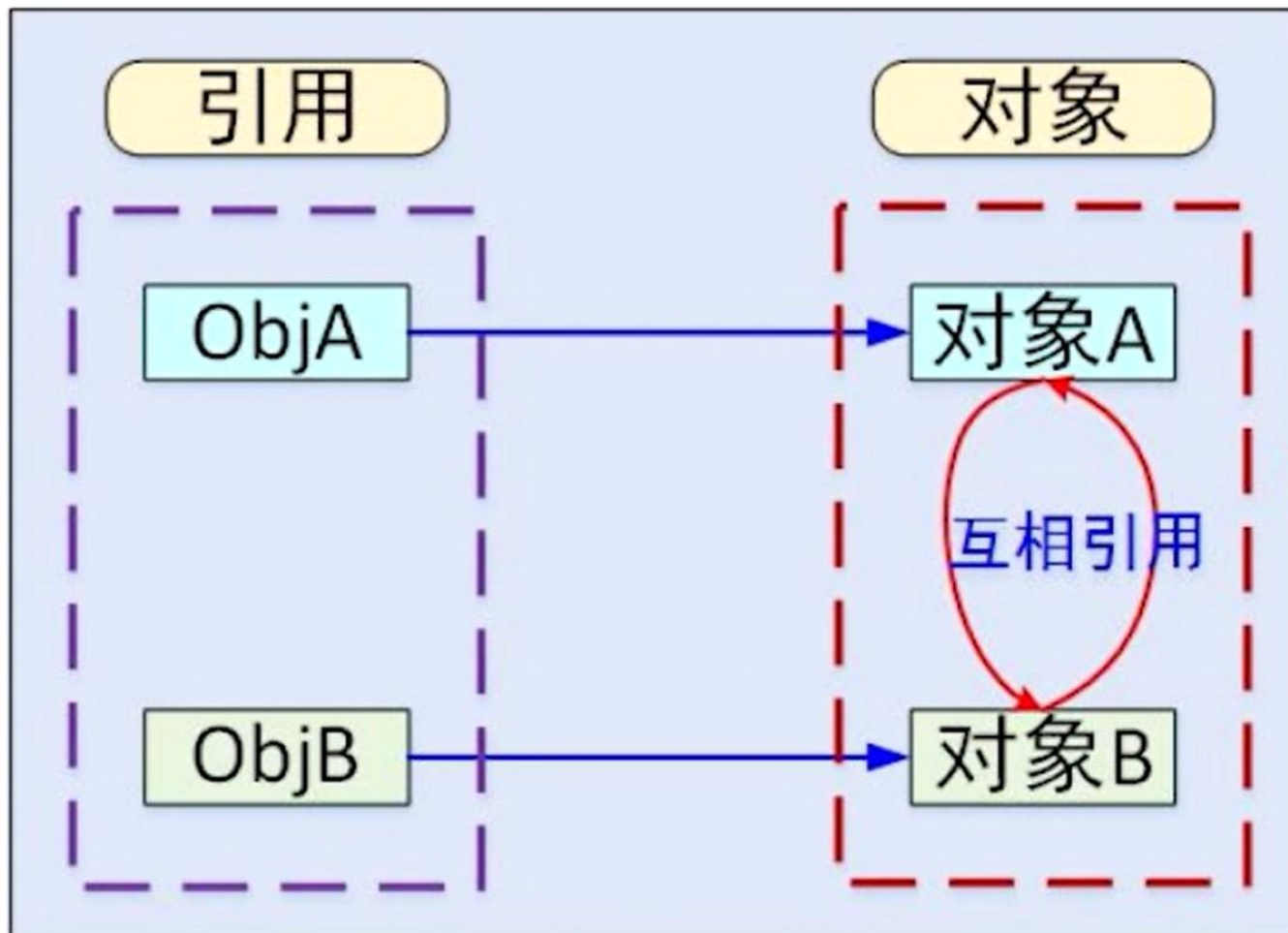
2023年3月30日 9:53

垃圾回收相关算法：

其二

1. 引用计数法

堆中的每个对象都对应一个引用计数器，当有引用指向这个对象时，引用计数器加1，而当指向该对象的引用失效时（引用变为null），引用计数器减1，最后如果该对象的引用计数器的值为0时，则Java垃圾回收器会认为该对象为无用对象并对其进行回收。优点是算法简单，缺点是“循环引用的无用对象”无法识别。



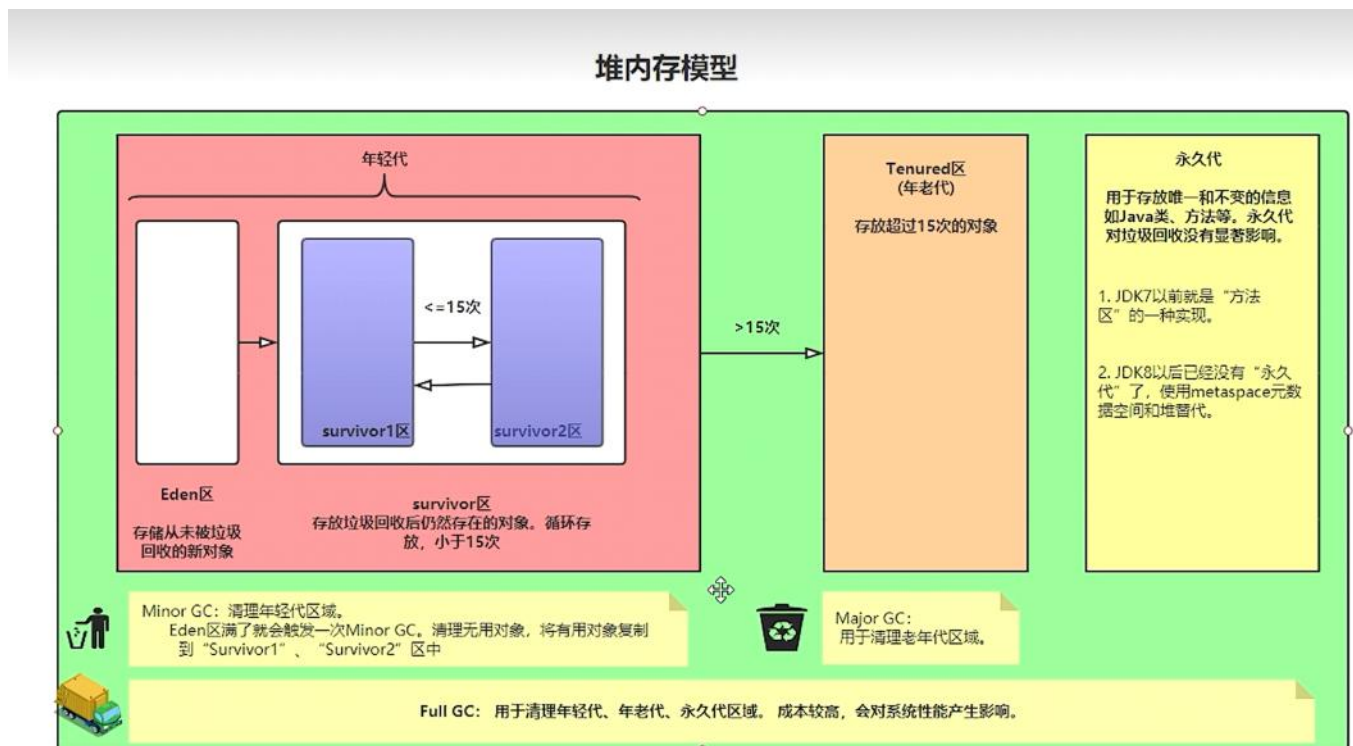
2. 引用可达法（跟搜索算法）

程序把所有的引用关系看作一张图，从一个节点GC ROOT开始，寻找对应的引用节点，找到这个节点以后，继续寻找这个节点的引用节点，当所有的引用节点寻找完毕后，剩余的节点则被认为是没有被引用到的节点，即无用的节点。

堆内存模型

2023年3月30日 10:36

堆内存模型



Eden区:存储了从未通过的垃圾回收的新对象

Survivor区:存放垃圾回收后, 仍然有用的对象, 循环存放, 小于15次垃圾回收次数都放在这个区里

Tenured区: 年老代区域存放超过15 次的垃圾回收的对象

Minor GC:

用于清理年轻代区域。Eden区满了就会触发一次Minor GC。清理无用对象, 将有用对象复制到"Survivor1"、"Survivor2"区中。

Major GC:

用于清理老年代区域。

Full GC:

用于清理年轻代、老年代区域。成本较高, 会对系统性能产生影响。

JVM调优和Full GC

在对JVM调优的过程中, 很大一部分工作就是对于Full GC的调节。有如下原因可能导致Full GC:

1. 年老代 (Tenured) 被写满
2. 永久代 (Perm) 被写满
3. `system.gc()`被显式调用
4. 上一次GC之后Heap的各域分配策略动态化

内存泄漏

2023年3月31日 17:18

如下四种情况最容易造成内存泄漏的场景

1. 创建大量无用对象

比如：大量的拼接字符串时，使用了String而不是StringBuilder。

```
String str = "";  
for (int i = 0; i < 10000; i++) {  
    str += i;    //相当于产生了 10000 个 String 对象  
}
```

2. 静态集合类的使用

像HashMap、Vector、List等的使用最容易出现内存泄漏，这些静态变量的生命周期和应用程序一致，所有的对象也不能被释放。

3. 各种连接对象（IO流对象、数据库连接对象、网络连接对象）未关闭

IO流对象、数据库连接对象、网络连接对象等连接对象属于物理连接，和硬盘或者网络连接，不使用的時候一定要关闭。

4. 监听器的使用不当

释放对象时，没有删除相应的监听器

其它要点

1. 程序员无权调用垃圾回收器
2. 程序员可以调用system.gc ()，该方法只是通知JVM，并不是运行垃圾回收器。尽量少用，会申请启动Full GC，成本高，影响系统性能。
3. Object对象的finalize方法，是Java提供给程序员用来释放对象或资源的方法，但是尽量少用。

包机制（package、import）

2023年3月31日 20:49

包（package相当于文件夹对于文件的作用。用于管理类、用于解决类的重名问题）。

Package的使用有两个要点：

- 1.通常是类的第一句非注释性语句。
- 2.包名：域名倒着写即可，便于内部管理类。

表 JDK 中的主要包	
Java 中的常用包	说明
java.lang	包含一些 Java 语言的核心类，如 String、Math、Integer、System 和 Thread。
java.awt	包含了构成抽象窗口工具集（abstract window toolkits）的多个类，这些类被用来构建和管理应用程序的图形用户界面(GUI)。
java.net	包含执行与网络相关的操作的类。
java.io	包含能提供多种输入/输出功能的类。
java.util	包含一些实用工具类，如定义系统特性、使用与日期日历相关的函数。

导入类 import

如果要使用其它包的类，需使用import，从而在本类中直接通过类名来调用，否则就需要书写类的完整包名和类名。

导入相同名称的类则需要写全完整包名和类名

Import java.until.* 导入该包下所有的类。会降低编译速度，但不会降低运行速度

静态导入

静态导入一下两种导入方法选择一种即可

Import static java.lang.Math.* 导入Math类的所有静态属性
Import static java.lang.Math.PI 导入Math类的PI属性

面对对象的三大特征（继承，封装，多态）

2023年3月31日 21:22

继承：

继承是面向对象编程的三大特征之一。继承让我们更加容易实现类的拓展。实现代码的重用，不用再重新发明轮子（don't reinvent wheels）

继承有两个主要作用：

- 1.代码复用，更加容易实现类的拓展
- 2.方便建模

继承 extends

Instanceof 运算符

Instanceof 是二元运算符左边是对象，右边是类；当对象是右面类或子类所创建对象时，返回true；否则，返回false。

继承使用要点

1. 父类也称超类、基类。子类：派生类等。
2. Java中只有单继承，没有像C++那样多继承。多继承会引起混乱，使得继承链过于复杂，系统难于维护。
3. Java中类没有多继承，接口有多继承。
4. **子类继承父类，可以得到父类的全部属性和方法（除了父类的构造方法），但不见得可以直接访问（比如，父类私有的属性和方法）。**
5. 如过定义一个类时，没有调用extends，则它的父类是：java.lang.Object

方法的重写Override

Override和overload 没有关系

子类重写父类的方法，可以用自身行为替换父类行为。重写是实现多态的必要条件。

方法重写需要符合下面的三个要点：

1. "==" :方法名、形参列表相同。
2. " <= "：返回值类型和声明异常类型，子类小于等于父类。
3. " >= "：访问权限，子大于等于父类。

Final关键字

final关键字的作用：

1. **修饰变量**：被他修饰的变量不可改变。一旦赋了初值，就不能被重新赋值。

```
Final int MAX_SPEED = 120;
```

2. **修饰方法**：该方法不可被子类重写。但是可以被重载！

```
Final void study(){}
```

3. **修饰类**：修饰的类不能被继承。比如：Math、String等。

```
Final class A{}
```

继承和组合：

组合比较灵活。继承只能有一个父类，但是组合可以有多个属性。所以，有人声称“组合优于继承，开发中可以不用继承”，但是，不建议大家走极端。

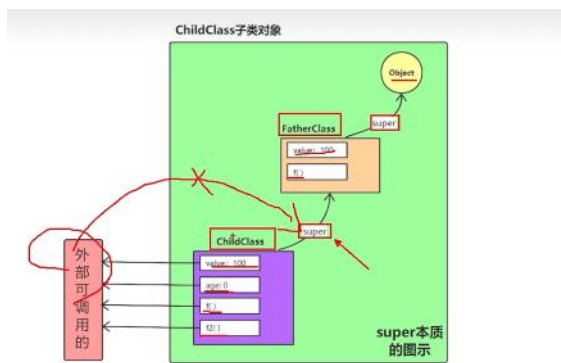
对于“is-a”关系建议使用继承，“has-a”关系建议使用组合。

比如：上面的例子，student is a Person 这个逻辑没问题，但是：Student has a Person就有问题了。这时候显然继承关系比较合适。

再比如：笔记本和芯片的关系显然是“has-a”关系，使用组合更好。

Super关键字

1. Super 可以看做是直接父类对象的引用。可通过super来访问父类中被子类覆盖的方法或属性。
2. 使用super调用普通方法，语句没有位置限制，可以在子类中随便调用。
3. 在一个类中，若是构造方法的第一行没有调用super(...)或者this(...)那么Java默认都会调用super()，含义是调用父类的无参数构造方法。



封装 (encapsulation)

编程中封装的具体优点：

提高代码的安全性。

提高代码的复用性。

“高内聚”：封装细节；便于修改内部代码，提高可维护性。

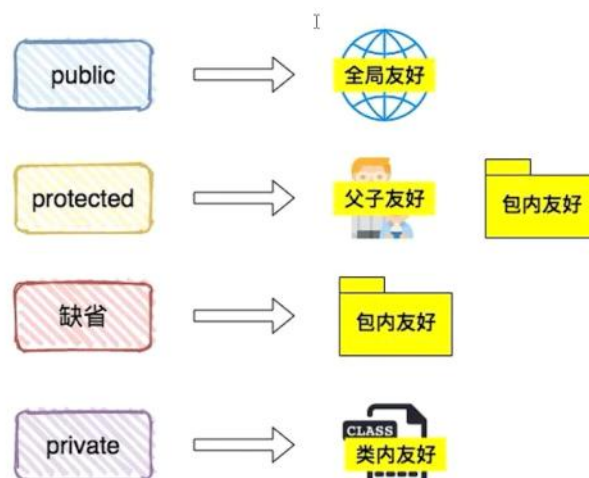
“低耦合”：简化外部调用，便于调用者调用，便于拓展和协作。

封装的实现——使用访问控制符

java是使用“访问控制符”来控制哪些细节需要封装，哪些细节需要暴露的。

java中4种“访问控制符”分别为private、default、protected、public

访问权限修饰符				
修饰符	同一个类	同一个包中	子类	所有类
private	☆			
default	☆	☆		
protected	☆	☆	☆	
public	☆	☆	☆	☆



【注】关于protected的两个细节：

1. 若父类和子类在同一个包中，子类**可访问**父类的protected成员，**也可访问**父类对象的protected成员。
2. 若子类和父类不在同一个包中，子类**可访问**父类的protected成员，**不能访问**父类对象的protected成员。

JAVABean

开发中封装的简单规则：

属性一般使用private访问权限。

属性私有后，提供相应的get/set方法来访问相关属性，这些方法通常是public修饰的，以提供堆属性的赋值与读取操作（注意：boolean变量的get方法是is开头！）。

方法：一些只用于本类的辅助性方法可以用private修饰，希望其它调用的方法用public修饰。

Set:设置

Get:获得

多态 (polymorphism)

多态指的是同一个方法调用，由于对象不同可能会有不同的行为。现实生活中具体实现会完全不同。比如：同样是调用人“吃饭”的方法，中国人用筷子吃，欧美用刀叉吃，印度人用手吃。

多态的要点：

1. 多态是方法的多态，不是属性的多态（多态于属性无关）。
2. 多态的存在要有3个必要条件：继承，方法重写，父类引用指向子类对象。
3. 父类引用指向子类对象后，用该父类引用调用子类重写的方法，此时多态就出现了。

Object

2023年4月1日 11:15

object类基本特性

- 1.Object 类是所有类的父类，所有的Java对象都拥有Object类的属性和方法。
- 2.如果在类的声明中未使用extends，则默认继承Object类。

补：IDEA部分快捷键

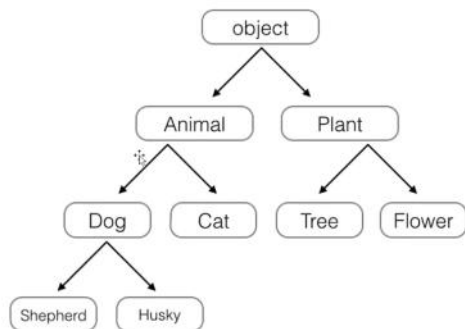
2023年4月1日 11:27

IDEA快捷键和相关操作：

1. 类的结构视图：alt+7
2. 看类的源码：ctrl+左键
3. 自动生成构造器、get、set方法、equals等：alt+insert
4. 查看错误：alt+enter
5. 快捷输出常见字符串：
 - a. Main public static void main(String[] args){}
 - b. Sout System.out.println();
 - c. Soutm System.out.println("描述：所在类中的，所在方法")

继承树的追溯

2023年4月2日 19:48



属性/方法查找顺序：（比如：查找变量h）

查找当前类中有没有属性h

依次上溯每个父类，查看每个父类中是否有h，直到Object

如果没找到，则出现编译错误。

上面步骤，只要找到h变量，则这个过程终止。

构造方法调用顺序：

构造方法第一句总是：super(...)来调用父类对应的构造方法。所以，流程就是：线向上追溯到Object，然后再依次向下执行类的初始化块和构造方法，直到当前子类为止。

注：静态初始化块调用顺序，与构造方法调用顺序一样，不再重复。

静态方法在程序运行时就生成。

JDBC

2023年4月5日 14:57

步骤:

注册驱动

```
Class.forName("com.mysql.jdbc.Driver");
```

获取驱动

```
Connection conn = DriverManager.getConnection(url,username,password);
```

定义SQL语句

```
String sql = "Update...";
```

获取执行SQL对象

```
Statement stmt = conn.createStatement();
```

执行SQL

```
Stmt.executeUpdate(sql);
```

DriverManager:

DriverManager(驱动管理类)作用:

1.注册驱动

```
Class.forName("com.mysql.cj.jdbc.Driver")
```

• 查看 Driver 类源码

```
static {
    try {
        DriverManager.registerDriver(new Driver());
    } catch (SQLException var1) {
        throw new RuntimeException("Can't register driver!");
    }
}
```

提示: MySQL5之后的驱动包, 可以省略注册驱动的步骤。

自动加载jar包中META-INF/services/java.sql.Driver文件中的驱动类

2.获取数据库连接

```
Static Connection getConnection(String url,String user,String password)
```

参数

1.url:连接路径

语法: jdbc:mysql://ip地址(域名): 端口号/数据库名称?藏书键值对1&参数键值对2...

实例: jdbc:mysql://127.0.0.1:3306/root

细节:

如果连接的是本机mysql服务器, 并且mysql服务器默认端口是3306, 则url可以简写为:jdbc:mysql:///数据库名称?参数键值对

配置 useSSL=false参数, 禁用安全连接方式, 解决警告提示

2.user:用户名

3.password:密码

Connection

1.获取执行SQL的对象

普通执行SQL对象

Statement createStatement()

预编译SQL的执行SQL对象:防止SQL注入

PreparedStatement PrepareStatement(sql)

执行存储过程的对象

CallableStatement PrepareCall(sql)

2.事务管理

MySQL事务管理

开启事物: BEGIN;/START TRANSACTION;

提交事物: COMMIT;

回滚事物: ROLLBACK;

MySQL默认自动提交事物

JDBC事物管理: Connection接口中定义了3个对应的方法

开启事务: setAutoCommit(boolean autoCommit):true为自动提交事务; false为手动提交事务。即为开启事务, 是在获取数据库连接对象后, 将其执为手动提交

提交事务: commit(): 执行未成功就回滚

回滚事务: rollback(): 执行成功后进行手动提交

事务:一个事件的完成需要好几个步骤完成后才算这个事情真正的完成。例如, 你用支付宝给别人转上一笔钱, 其操作的完成是, 你给他转一笔钱后, 1.你的支付上面显示少了500, 2.然后他的支付宝上显示多了500, 此时这个事件才算完成。而转钱就是一个事务, 其中的1和2是它的两个步骤。假如其中有那个显示未更新就会出现错误, 那么如何避免? ? ? ?

DDL:对库和表的增删改查操作

DML:对数据的增删改操作

DQL: 对数据的查询操作

Statement

Statement作用:

1.执行SQL语句

执行SQL语句

Int executeUpdate(sql):执行DML、DDL语句

返回值: (1) DML语句应影响的行数 (2) DDL语句执行后, 执行成功也可能返回0

ResultSet executeQuery(sql):执行DQL语句

返回值: ResultSet结果集对象

获取查询结果:

Boolean next():(1)将光标从当前位置向前移动一行

(2)判断当前行是否为有效行

返回值:

True:有效行, 当前行有数据

False:无效行, 当前行没有数据

XXX getxxx(参数): 获取数据

Xxx:数据类型; 如: int getInt(参数):String getString(参数)

参数:

Int:列的编号, 从1开始

String:列的名称

```
ResultSet rs = stmt.executeQuery(sql);

while(rs.next()){
    int id = rs.getInt( columnLabel: "id");
    String name = rs.getString( columnLabel: "mname");
    System.out.println(id);
    System.out.println(name);
    System.out.println("-----");
}
```

SQL注入: 'or'1='1

在数据库中会呈现成 `select * from zh where username = '"+name+"' and password = 'or'1='1'`

PreparedStatement

PreparedStatement作用: 预编译SQL并执行SQL语句

获取 PreparedStatement 对象

```
String sql = "select * from user where username = ? And password = ?" ;
```

```
PreparedStatement pstmt = con.prepareStatement(sql);
```

设置参数值:

PreparedStatement对象: setXXX(参数1, 参数2): 给? 赋值

xxx: 数据类型; 如setint (参数1, 参数2)

参数:

参数1: ?的位置编号, 从1开始

参数2: ? 的值

执行SQL:

`executeUpdate(); excuteQuery();`不需要再传递sql

1.`execute(String query)`方法用来执行任意的SQL语句查询, 如果查询的结果是一个ResultSet,这个方法就返回true,如果结果不是ResultSet, 比如: insert或update, 就会返回false.可以通过`getResultSet`方法来获取ResultSet。

2.`executeQuery(String query)`接口用来执行select查询, 并返回ResultSet,即便查询不到记录, 返回的ResultSet也不会为null.

3.`executeUpdate(String query)`方法用来执行insert, delete或者update语句, 返回值是int类型, 如果是DML语句, 则是更新的条数, 如果是DDL, 则返回0

PreparedStatement 预编译功能开启: `useServerPrepStmts=true`

配置MySQL执行日志 (重启软件服务后生效)

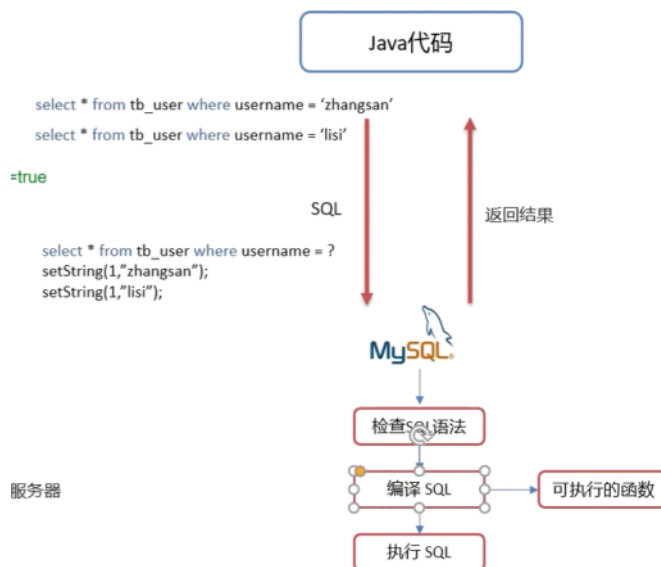
```
log-output=FILE
general-log=1
general_log_file="D:\mysql_slow.log"
slow-query-log=1
slow_query_log_file="D:\mysql_slow.log"
long_query_time=2
```

原理：

在获取PreparedStatement对象时，将sql语句发送给mysql服务器进行检查，编译（这些步骤很耗时）

执行时就不用再进行这些步骤了，速度更快

如果sql模板一样，则只需要进行以此检查，编译



单例模式

2023年4月7日 18:41

单例模式的两种模式：

饿汉模式

懒汉模式

饿汉模式：

GiantDragon 应该只有一只，通过私有化其构造方法，使得外部无法通过new 得到新的实例。

GiantDragon 提供了一个public static的getInstance方法，外部调用者通过该方法获取12行定义的对象，而且每一次都是获取同一个对象。从而达到单例的目的。

这种单例模式又叫做饿汉式单例模式，无论如何都会创建一个实例

```
1 package Mypackage;
2
3 import java.util.Scanner;
4
5 7 usages
6 public class Hero {
7     1 usage
8     private Hero(){
9         System.out.println(1111);
10    }
11
12    1 usage
13    private static Hero instance = new Hero();
14
15    2 usages
16    public static Hero getInstance (){
17        return instance;
18    }
19 }
```

懒汉模式：

懒汉式单例模式与饿汉式单例模式不同，只有在调用getInstance的时候，才会创建实例

```
1 package Mypackage;
2
3 import java.util.Scanner;
4
5 7 usages
6 public class Hero {
7     1 usage
8     private Hero(){
9         System.out.println(1111);
10    }
```

```

1  package Mypackage;
2
3  import java.util.Scanner;
4
5  7 usages
6  public class Hero {
7      1 usage
8      private Hero(){
9          System.out.println(1111);
10     }
11
12     3 usages
13     private static Hero instance = null;
14
15     2 usages
16     public static Hero getInstance (){
17         if (null == instance){
18             instance = new Hero();
19         }
20         return instance;
21     }
22 }

```

网址参考: <https://how2j.cn/k/class-object/class-object-singleton/349.html#nowhere>

Enum

2023年4月7日 19:00

枚举enum是一种特殊的类(还是类), 使用枚举可以很方便的定义常量
比如设计一个枚举类型 季节, 里面有4种常量

用法:

```
public class Hero {
    public enum HeroType {
        TANK,WIZARD,ASSASSIN,ASSIST,WARRIOR,RANGED,PUSH,FARMING;
    }

    public static void main(String[] args) {
        HeroType type = HeroType.TANK;
        for (HeroType s :HeroType.values()) {
            System.out.println(s);
        }
        // switch(type){
        //     case TANK:
        //         System.out.println("坦克");
        //     case WIZARD:
        //         System.out.println("法师");
        //     case ASSASSIN:
        //         System.out.println("刺客");
        //     case ASSIST:
        //         System.out.println("辅助");
        //     case WARRIOR:
        //         System.out.println("近战");
        //     case RANGED:
        //         System.out.println("远程");
        //     case PUSH:
        //         System.out.println("推进");
        //     case FARMING:
        //         System.out.println("打野");
        // }
    }
}
```

一个标准JavaBean需要

2023年4月25日 12:16

1. 私有化成员变量
2. 空参构造方法
3. 带全部参数的构造方法
4. Get/set方法

ThreadLocal

2025年7月16日 19:24

ThreadLocal是JAVA并发包中提供的一个类，它的主要作用是在多线程环境下为每个线程提供一个独立的变量副本，使得每个线程在访问ThreadLocal时获取到的都是自己的私有变量，而不是共享的同一个变量。

ThreadLocal的出现主要是为了解决多线程环境下的数据共享问题。在传统的多线程编程中，多个线程之间共享数据通常是通过共享对象来实现的。但是这种方式在处理多个线程之间需要共享大量数据时，会带来一些问题。

- 多个线程同时修改共享数据时可能会出现竞争条件（race condition），导致数据的不一致性。
- 如果多个线程需要访问共享数据，就需要进行频繁的同步操作，这会降低程序的性能

通过使用ThreadLocal，可以将需要共享的数据存储在每个线程的本地变量中，每个线程只能看到和修改自己的副本，而不会影响到其他线程的副本，这样就可以避免多个线程同时修改同一份数据，避免了竞争条件和数据不一致性的问题。同时，由于每个线程都有自己的数据副本，不需要进行频繁的同步操作，提高了程序的性能。

应用场景

1. 线程上下文信息传递：例如在web应用中，服务器接收到请求后，需要在不同的过滤器、处理器链路中传递用户会话信息，此时可以将这些信息存放在ThreadLocal中。因为在Servlet容器中，每个HTTP请求都会被分配到一个单独的线程中处理。
2. 避免同步开销：对于哪些只需要在单个线程内保持状态，不需要线程间共享的数据，使用ThreadLocal可以避免使用锁带来的性能损耗。
3. 数据库连接、事务管理：在多线程环境下，每个线程有自己的数据库连接，可以使用ThreadLocal存储当前线程的数据库连接对象，以确保线程安全。
4. 线程安全类：对于一些不是线程安全的类（如SimpleDateFormat），可以通过ThreadLocal为每个线程提供独立的实例，从而避免线程安全问题

接口方法

Void set(T value)：设置当前线程的线程局部变量的值

Public Object get()：该方法返回当前线程所对应的线程局部变量

Public void remove()：将当前线程局部变量的值删除，目的时为了减少内存的占用

Protected Object initialValue()：返回该线程局部变量的初始值，该方法是一个Protected的方法，显然是为了让子类覆盖而设计的。这个方法是一个延迟调用方法，在线程第一次条用get()或set()时才执行，并且仅执行1次。ThreadLocal中的默认实现

直接返回一个null

Lambda

2025年9月18日 19:59

lambda表达式是JDK8开始后的一种信誉发形式。

函数式接口(有且仅有一个抽象方法的接口，可用@FunctionalInterface装饰)，可以用于替代某些匿名内部类对象，从而让程序更简洁，可读性更好。

(被重写方法的形参列表) -> {
被重写方法的方法体代码。
}

注意：Lambda表达式只能替代函数式接口的匿名内部类!!!

```
public class lambad {  
    public static void main(String[] args) {  
        Animal a = new Animal(){  
            @Override 1个用法  
            public void cry() {  
                System.out.println("666");  
            }  
        };  
  
        a.cry();  
  
        Swim s = () ->{  
            System.out.println("22");  
        };  
  
        s.swimming();  
    }  
}  
  
abstract class Animal { 2个用法 1个继承者  
    public abstract void cry (); 1个用法 1个实现  
}  
  
// 函数式接口，有且仅有一个抽象方法的接口  
@FunctionalInterface // 声明函数式接口 1个用法  
interface Swim{  
    void swimming(); 1个用法  
}
```

Lambda表达式只能替代函数式接口的匿名内部类

```

4      import java.util.Comparator;
5
6      public class LambdaDemo2 {
7          public static void main(String[] args) {
8              Integer[] arr = {10, 9, 5, 4, 3, 2, 1};
9
10             //      Arrays.sort(arr, new Comparator<Integer>() {
11             //          @Override
12             //          public int compare(Integer o1, Integer o2) {
13             //              return o1 - o2;
14             //          }
15             //      });
16             //
17             //      Arrays.sort(arr, (Integer o1, Integer o2)-> {
18             //          return o1 - o2;
19             //      });
20
21             // 省略写法
22             Arrays.sort(arr, ( Integer o1, Integer o2)-> o1 - o2);
23
24             System.out.println(Arrays.toString(arr));
25         }
26     }
27

```

只要代码的上文可推导，那么都可以省略

ArrayList集合的基本使用

2023年4月24日 21:53

数组的缺点长度固定
集合和数组一样都是个容器但是集合的长度可扩容
数据可以存基本数据类型与引用数据类型
集合不能直接存基本数据类型，如果要存基本类型要将它们变为**包装类**。
基本数据类型对应的包装类

byte	Byte
short	Short
char	Character
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

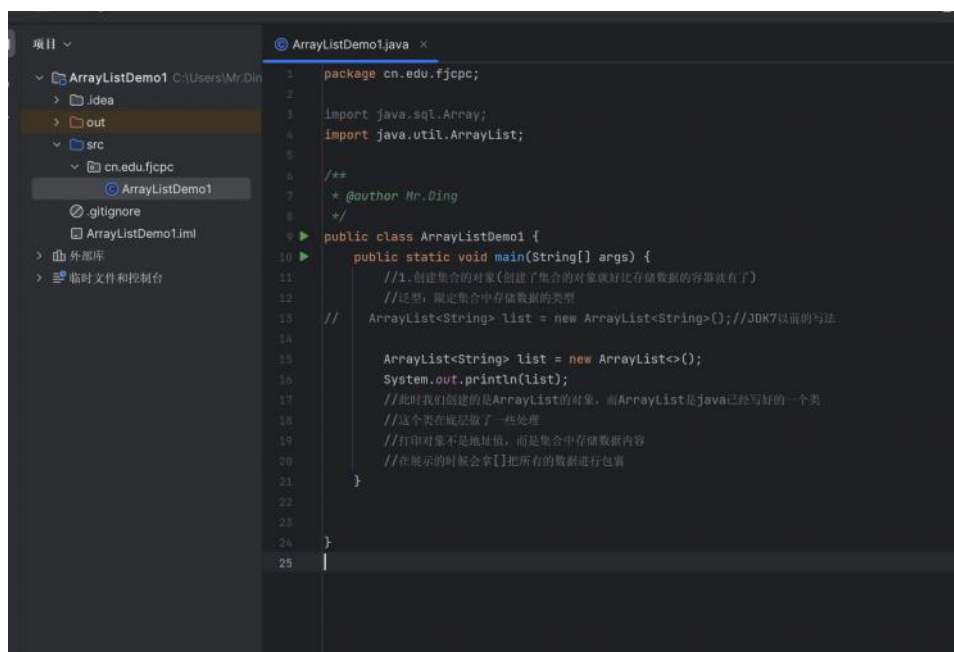
```
int[] arr1 = new int[3];  
  
User u1 = new User("小红",23);  
User u2 = new User("小明",23);  
User[] userArr = new User[2];  
userArr[0] = u1;  
userArr[1] = u2;
```

可以存基本数据类型

可以存引用数据类型

自动扩容：集合的长度根据添加的元素自动增长（自动伸缩可长可短）

创建第一个ArrayListDemo



第一步创建集合的对象（创建了集合的对象就好比存储数据的容器就有了）

如何像数组中一样定义存储数据的类型呢？利用泛型

泛型：限定集合中存储数据的类型

JDK7以前的创建集合对象的写法:ArrayList<String> list = new ArrayList<String>() String是引用类型注解

ArrayList<String> list = new ArrayList<>();

这是现在创建集合对象的写法，后面的尖括号内不需要写类型但是需要保留尖括号

System.out.println(list);

此时我们创建的是ArrayList的对象，而ArrayList是java已经写好的一个类，这个类在底层做了一些处理，打印对象不是地址值，而是集合中的存储数据内容，在展示的时候会拿[]把所有的数据进行包裹

ArrayList成员方法

增：boolean add(E e)	添加元素，返回值表示是否添加成功
删：boolean remove(E e)	删除指定元素，返回值表示是否删除成功
E remove(int index)	删除指定索引的元素，返回被删除元素
改：E set(int index,E e)	修改指定索引下的元素，返回原来的元素
查：E get(int index)	获取指定索引的元素
获取长度：Int size()	集合的长度，也就是集合中的元素个数

	方法名	说明
增	boolean add(E e)	添加元素, 返回值表示是否添加成功
	boolean remove(E e)	删除指定元素, 返回值表示是否删除成功
删	E remove(int index)	删除指定索引的元素, 返回被删除元素
	E set(int index, E e)	修改指定索引下的元素, 返回原来的元素
改	E get(int index)	获取指定索引的元素
查	int size()	集合的长度, 也就是集合中元素的个数

Add:无论添加了什么进去就会返回布尔型True, 所以一般不管它的返回值, 直接调用方法, 使用方法本身就行了。

```

7 // //增
8 list.add("aaa");
9 list.add("bbb");
10 list.add("ccc");
11 //// 删
12 // list.remove("aaa");
13 // list.remove(2);
14 // 改
15 // list.set(0, "ddd");
16 // 查
17 // String s = list.get(2);
18 // System.out.println(s);
19 // 获取长度
20 // int size = list.size();
21 // System.out.println(size);
22 // 遍历集合
23 for (int i = 0; i < list.size(); i++) {
24     String s = list.get(i);
25     System.out.println(s);
26 }

```

Collections常用的API

方法名称	说明
public static <T> boolean addAll (Collection<T> c, T... elements)	批量添加元素
public static void shuffle (List<?> list)	打乱List集合元素的顺序
public static <T> void sort (List<T> list)	排序
public static <T> void sort (List<T> list, Comparator<T> c)	根据指定的规则进行排序
public static <T> int binarySearch (List<T> list, T key)	以二分查找法查找元素
public static <T> void copy (List<T> dest, List<T> src)	拷贝集合中的元素
public static <T> int fill (List<T> list, T obj)	使用指定的元素填充集合
public static <T> void max/min (Collection<T> coll)	根据默认的自然排序获取最大/小值
public static <T> void swap (List<?> list, int i, int j)	交换集合中指定位置的元素

什么是代理

2025年10月20日 10:36

代理的目的是为了避免侵入式修改，在项目中不能修改原来的代码，又要增加额外的功能，这个时候就可以用到代理，代理类似中介公司

无侵入式的给代码增加新的功能

1. 为什么需要代理？

代理可以无侵入式的给对象增强其他的功能



2. 代理长什么样？

代理里面就是对象要被代理的方法

3. Java通过什么来保证代理的样子？

通过接口保证，后面的对象和代理需要实现同一个接口
接口中就是被代理的所有方法